

CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN METHODOLOGY

3.1 Introduction

This chapter presents the analysis and design work that produced CampusBites, the online food ordering system built for this project. CampusBites is a role-based web application that allows students at University of Africa, Toru-Orua (UAT) to order food from campus vendors without having to queue physically at an eatery, allows vendors to receive and manage those orders from a dashboard, and allows an administrator to control which vendors are allowed to trade on the platform. The chapter covers how the existing (manual) way of getting food on campus was analysed, the problems that analysis surfaced, the methodology adopted to build a replacement, the functional and non-functional requirements derived from that analysis, and the design artefacts — architecture, database design and UML diagrams — that guided the implementation described in Chapter Four.

Everything described in this chapter reflects the system as it was actually built. Where the implementation departs from what might be expected of a typical academic food-ordering system — for instance, the choice of a JSON document store instead of a relational database, or the decision to poll for order updates instead of using WebSockets — the reasoning behind that decision is stated explicitly rather than glossed over.

3.2 Analysis of Existing System

Before CampusBites, food ordering at UAT’s campus eateries followed the pattern common to most Nigerian tertiary institutions: a student walks to a vendor’s stall or shop, joins whatever queue is there, states their order verbally, waits while it is prepared (or while orders ahead of theirs are prepared), and pays on collection. Some vendors keep a rough mental or paper record of pending orders during busy periods; none of the vendors observed during this project used any digital system to record, sequence, or track orders.

This arrangement works, in the sense that food gets served, but it has no mechanism for a student to know how long a wait will be, no way for a vendor to smooth out a rush of simultaneous orders, and no institutional oversight of who is operating as a food vendor on campus. Vendors range from established eateries to informal student-run “food hustles” operating out of a room or a table, and there is no shared, campus-wide way for a new vendor to be discovered by students or for the school to keep a record of who is currently trading.

3.3 Problems Identified in the Existing System

Analysis of the existing (manual) process identified the following recurring problems:

1. **Physical queueing.** Students must be physically present at a vendor to place and wait for an order, which is wasteful of time between lectures.
2. **No order status visibility.** Once an order is placed verbally, the student has no way of checking its progress except by waiting at the counter or asking again.
3. **Order mix-ups during rush periods.** Verbal ordering with no written or digital record makes it easy for a vendor serving several customers at once to lose track of who ordered what.
4. **No discovery mechanism for smaller vendors.** Student-run food hustles rely entirely on word of mouth; there is no shared listing a new student can browse to see what is available on campus.
5. **No vetting of who can sell food on campus.** Anyone can claim to be selling food with no institutional record of active, approved vendors.
6. **No historical record for the student.** There is no order history a student can refer back to — for a repeat order, a receipt, or a record of what was actually paid for a given item on a given day.

3.4 Justification for the New System

CampusBites was built specifically to address the problems listed in Section 3.3, rather than to replicate the feature set of a general-purpose commercial food delivery app. Because the existing process is pickup-based rather than delivery-based, the system was scoped around **ordering ahead and tracking status**, not around delivery logistics, live rider tracking, or payment processing — none of which the existing campus process has or needs at this stage.

Concretely, the new system addresses each problem as follows: students can browse a vendor's menu and place an order from anywhere with a browser, removing the need to queue before ordering (Problem 1); the order-tracking page polls the server every four seconds so a student can see their order move from “received” to “preparing” to “ready” without needing to ask anyone (Problem 2); every order is recorded as a structured record with itemised quantities and a price snapshot at the time of ordering, removing the ambiguity of a verbal order (Problem 3); every vendor, from an established eatery to a one-person snack hustle, gets the same listing page a student can browse (Problem 4); a vendor account is not usable until an administrator has approved it, giving the school a single point of control over who can trade (Problem 5); and a student's past orders remain visible on an order history page (Problem 6).

3.5 Methodology Adopted

The system was built iteratively rather than to a single upfront design. Development began by deliberately narrowing scope to the core order flow — registration, login, browsing, ordering, and status tracking — before any visual design work was attempted, so that a working, clickable system existed early. Later iterations layered additional work on top of that working core: a full visual redesign of every page around a single dark colour theme and a consistent component system, followed by the addition of real vendor and menu photography, and the addition of five further vendors to the seed data. Each iteration was tested against the running application before the next one began, rather than all requirements being designed in full up front and implemented in one pass.

This incremental approach was chosen over a strict Waterfall model because the requirements for a project like this are refined by seeing the system run — for example, the decision to compile Tailwind CSS locally rather than load it from a public CDN (discussed in Section 4.3) was made only after the CDN-based approach was seen to fail in the actual deployment environment, something that would not have surfaced from design documents alone. It is also worth recording candidly that the project’s original specification document anticipated SQLite (via `better-sqlite3`) as the persistence layer; the system as actually implemented instead uses a JSON document file (`data/db.json`) managed through a small custom module (`data/store.js`). This is exactly the kind of adjustment an iterative approach is meant to absorb: the simpler flat-file store was sufficient for the scope actually being built and removed a native-module dependency, so the plan was revised rather than followed rigidly.

3.6 Requirements Analysis

3.6.1 Functional Requirements

The functional requirements below were derived directly from the three user roles the system supports — student, vendor and administrator — and correspond one-to-one with routes implemented in `routes/auth.js`, `routes/student.js`, `routes/vendor.js` and `routes/admin.js`.

Table 3.1: Functional Requirements

No.	Requirement	Implemented by
FR1	A visitor shall be able to register as either a student or a vendor, supplying role-specific fields (business name and description for vendors).	POST /register
FR2	A registered user shall be able to log in with an email and password, and log out.	POST /login, POST /logout
FR3	A newly registered vendor account shall remain inactive (“pending”) until an administrator approves it.	status field on users; enforced at login in routes/auth.js
FR4	A logged-in student shall be able to browse a list of active (approved) vendors.	GET /vendors
FR5	A student shall be able to view a specific vendor’s menu with item names and prices.	GET /vendors/:id
FR6	A student shall be able to build a cart of menu items and quantities before ordering.	Client-side cart logic in views/vendor-menu.ejs
FR7	A student shall be able to submit the cart as an order, which is recorded with an itemised price snapshot.	POST /orders
FR8	A student shall be able to view their own order	GET /orders, GET /api/orders/mine

No.	Requirement	Implemented by
	history with a live-updating status.	
FR9	A vendor shall be able to view their own incoming, unfulfilled orders.	GET /vendor/dashboard, GET /api/vendor/orders
FR10	A vendor shall be able to advance an order through its status lifecycle one step at a time.	POST /vendor/orders/:id/advance
FR11	An administrator shall be able to view platform-wide counts (students, active vendors, pending vendors, total orders).	GET /admin/dashboard
FR12	An administrator shall be able to approve or reject a pending vendor account.	POST /admin/vendors/:userId/approve, POST /admin/vendors/:userId/reject
FR13	Rejecting a vendor shall remove that vendor's business record and menu items along with it.	routes/admin.js reject handler

3.6.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

No.	Requirement	How it is addressed
NFR1	Usability	A single, consistent dark-themed design system (colour palette, spacing, typography, reusable component classes) is applied across every page so the interface behaves predictably regardless of which role is using it.
NFR2	Responsiveness	Layouts use a mobile-first grid (Tailwind CSS breakpoints) so pages remain usable on phone, tablet and desktop widths.
NFR3	Security	Passwords are hashed with bcrypt before storage; no plaintext password is ever persisted. Role-based access is enforced server-side by the <code>requireRole()</code> middleware, not merely hidden in the interface.
NFR4	Data integrity	The price of each order item is copied into the order record (<code>priceAtOrder</code>) at the moment of ordering, so a later change to a menu item's price cannot alter the amount owed on an existing order.
NFR5	Availability of core styling	The visual design does not depend on a third-party CDN being reachable at runtime; Tailwind CSS is compiled to a local file served by the application itself (see Section 4.3).

No.	Requirement	How it is addressed
NFR6	Maintainability	Vendor and menu seed data is defined once in <code>data/vendor-catalog.js</code> and consumed by both the full reseed script and the incremental “add vendors” script, so the two never drift out of sync.
NFR7	Graceful degradation	A missing vendor or menu photo does not break a page; the interface falls back to an icon placeholder instead of showing a broken image.

3.7 System Design

CampusBites follows a layered, server-rendered architecture rather than a single-page-application design, in keeping with its scope as a project built by one developer without a separate front-end build pipeline. The layers are:

- **Presentation layer** — EJS templates in `views/`, split into page templates and two shared partials (`views/partials/header.ejs`, `views/partials/footer.ejs`) that every page includes, so the navigation bar, footer, fonts, icon library and compiled stylesheet only need to be defined once.
- **Routing / controller layer** — four Express routers (`routes/auth.js`, `routes/student.js`, `routes/vendor.js`, `routes/admin.js`), each scoped to one area of the system and mounted on the main `server.js` application.
- **Middleware layer** — `middleware/auth.js` exposes `attachUser` (reads the session and exposes the current user to every view) and `requireRole` (blocks a request unless the session’s user has one of the allowed roles).
- **Data / persistence layer** — `data/store.js` exposes `load()`, `save()` and a `tx()` helper that loads the JSON file, lets a callback mutate it, and writes it back, giving the rest of the application a single, synchronous place where all reads and writes happen.
- **Client-side behaviour** — small, page-scoped `<script>` blocks (no front-end framework) handle the cart on the vendor menu page and the four-second polling used on the order-tracking and vendor dashboard pages.

The request/response cycle is conventional Express: a request enters through session and body-parsing middleware, is attached the current user, is routed to the matching handler, and that handler either renders an EJS view or returns JSON (for the two polling endpoints, `/api/orders/mine` and `/api/vendor/orders`). Static assets — the compiled stylesheet and vendor/menu photos — are served directly from the `public/` directory by Express's built-in static file middleware.

3.8 Database Design

The system's data is persisted as a single JSON document (`data/db.json`) containing five collections — `users`, `vendors`, `menuItems`, `orders` and `orderItems` — plus a `counters` object used to generate sequential integer IDs. Although the storage mechanism is a flat file rather than a relational database engine, the five collections are still designed relationally: each record carries foreign-key-style references to the record it belongs to, and the relationships between them are exactly what would be expressed as foreign keys in a SQL schema.

Table 3.3: users

Field	Type	Description
id	integer	Primary key
name	string	Full name
email	string	Login identifier, must be unique
passwordHash	string	bcrypt hash of the password (cost factor 8)
role	enum	student vendor admin
status	enum	active pending rejected (vendors only meaningfully use pending/rejected)
createdAt	ISO datetime	Account creation timestamp

Table 3.4: vendors

Field	Type	Description
id	integer	Primary key
userId	integer	Foreign key → <code>users.id</code> (the account that owns this vendor profile)
businessName	string	Public-facing business name
description	string	Short description shown on the vendor listing and menu page

Table 3.5: menuItems

Field	Type	Description
id	integer	Primary key
vendorId	integer	Foreign key → <code>vendors.id</code>
name	string	Dish or product name
price	number	Price in naira

Table 3.6: orders

Field	Type	Description
id	integer	Primary key
studentId	integer	Foreign key → <code>users.id</code> (the student who placed the order)
vendorId	integer	Foreign key → <code>vendors.id</code>
status	enum	pending → preparing → ready → completed
createdAt	ISO datetime	Order placement timestamp

Table 3.7: orderItems

Field	Type	Description
id	integer	Primary key
orderId	integer	Foreign key → orders.id
menuItemId	integer	Foreign key → menuItems.id
quantity	integer	Quantity ordered
priceAtOrder	number	Snapshot of the menu item's price at the moment the order was placed

The relationships between these six tables are shown as a class diagram in Figure 3.2 (Section 3.9.2): a user owns at most one vendor profile; a user (as a student) places many orders; a vendor offers many menu items and receives many orders; and an order contains many order items, each of which references the menu item it was ordered from.

3.9 Unified Modeling Language (UML) Diagrams

3.9.1 Use Case Diagram

Figure 3.1 shows the use cases available to each of the three actors. Login/Logout is shown once, under Student, to avoid repeating an identical use case three times; all three roles authenticate through the same mechanism, and which further use cases a logged-in session can reach is decided by the `requireRole()` middleware rather than by the interface.

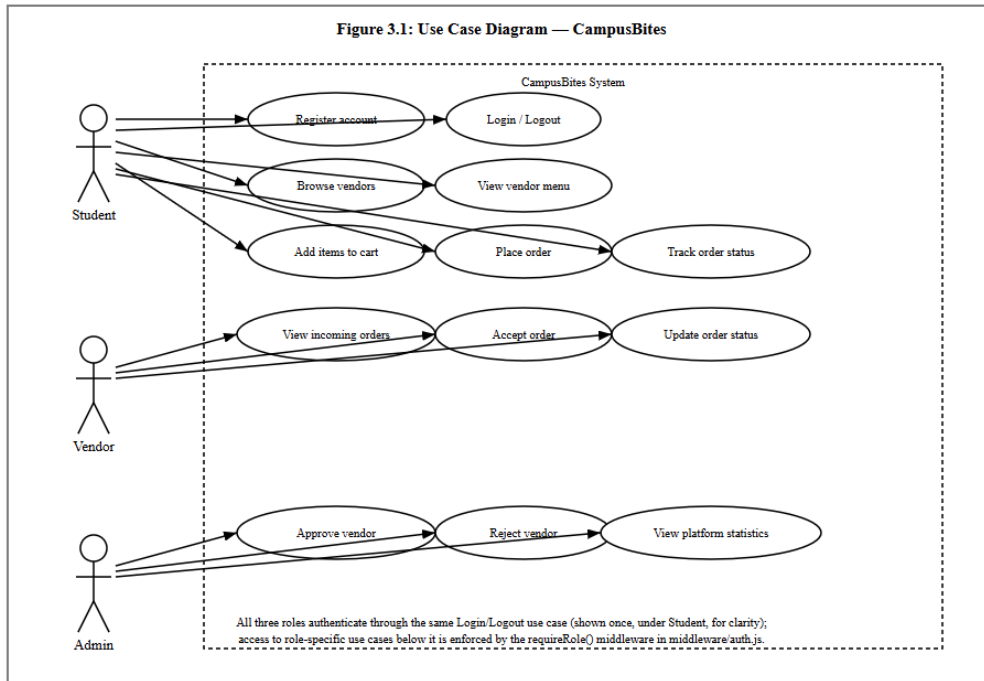


Figure 3.1: Use Case Diagram

3.9.2 Class Diagram

Figure 3.2 models the five persisted entities described in Section 3.8 as classes, with the multiplicities of the relationship between each pair.

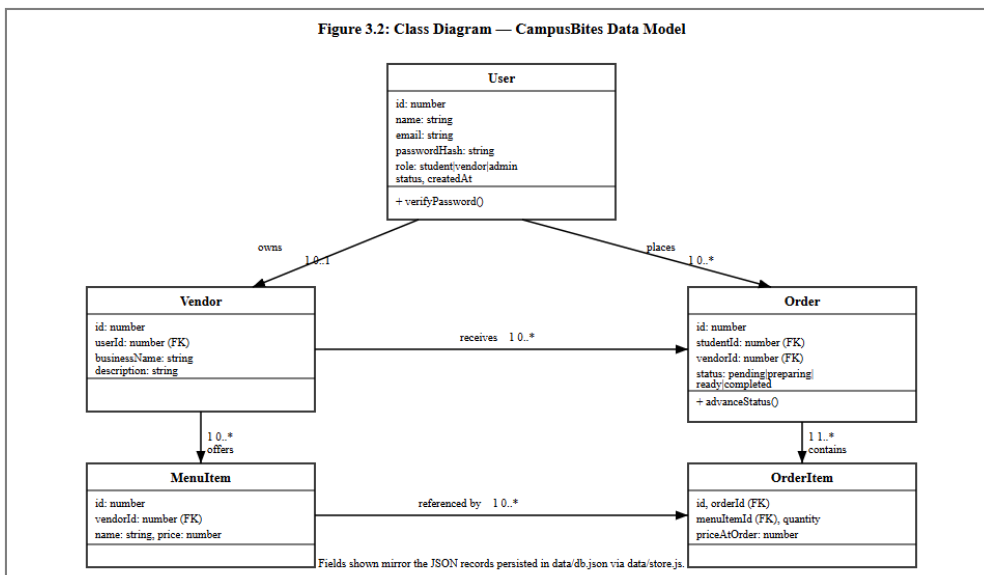


Figure 3.2: Class Diagram

3.9.3 Sequence Diagram

Figure 3.3 traces a single, representative interaction — a student placing an order — across the four collaborating parts of the system: the browser, the client-side cart script, the Express route handling `POST /orders`, and the `store.js` persistence module. It reflects the actual control flow in `routes/student.js`, including the price-snapshot step and the subsequent polling that populates the order-tracking page.

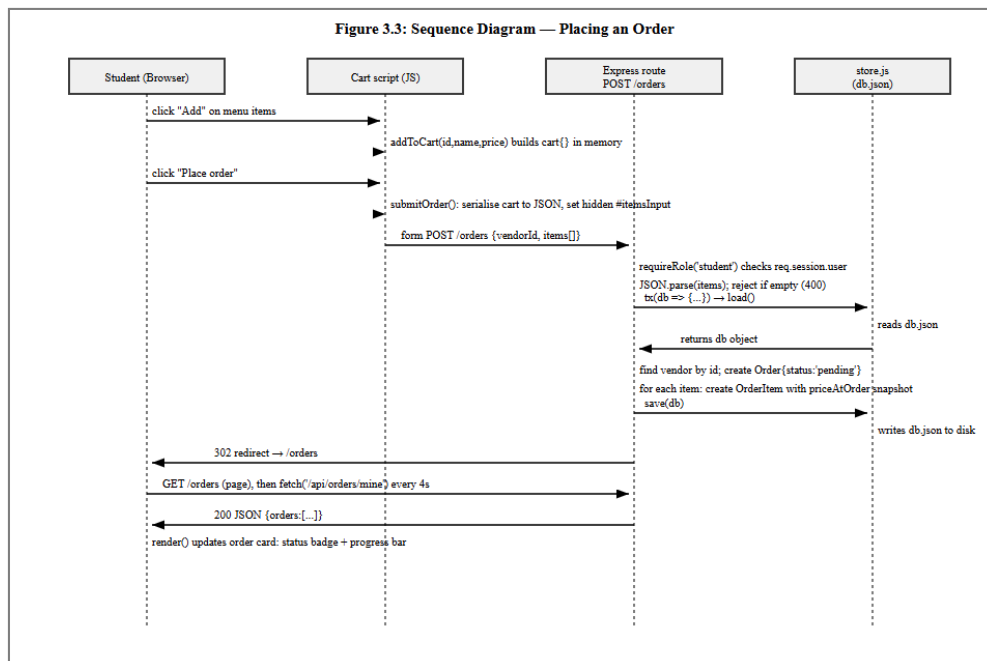


Figure 3.3: Sequence Diagram

3.9.4 Activity Diagram

Figure 3.4 models the order status lifecycle itself, from the moment a student places an order to the moment it is marked completed. The dotted branch is included deliberately to record a real, current limitation: there is no cancellation activity implemented, so an order left unattended by a vendor simply remains in the “pending” state indefinitely.

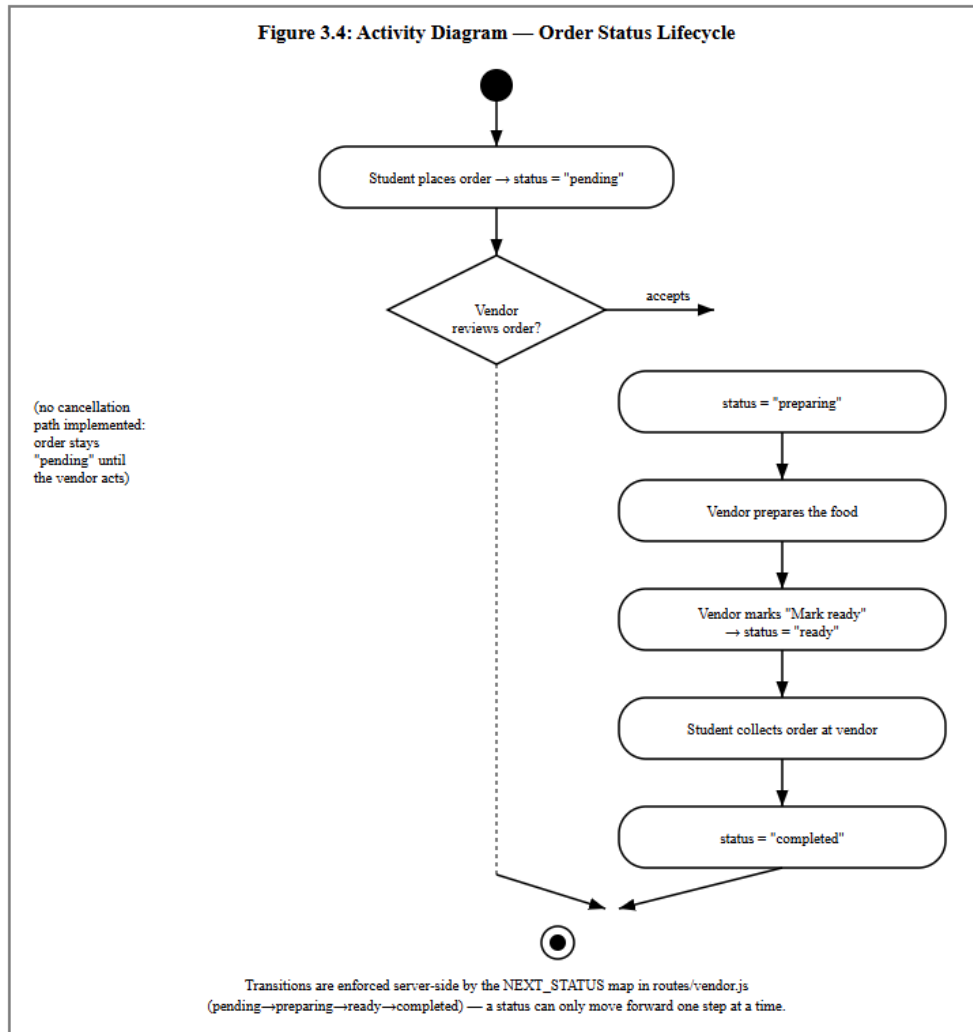
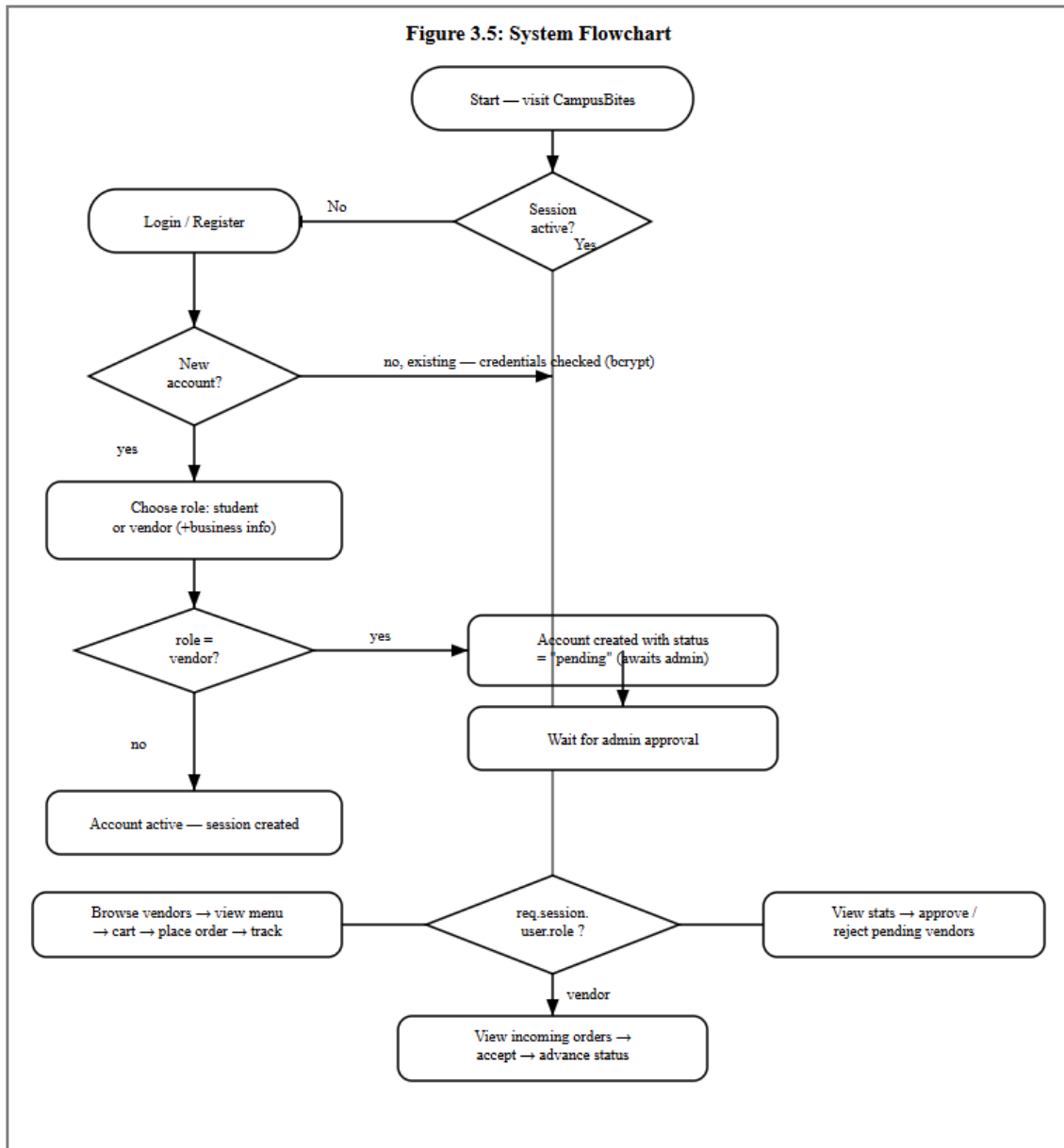


Figure 3.4: Activity Diagram

3.10 System Flowchart

Figure 3.5 gives an overall, role-agnostic view of how a single visit to the system is handled, from the initial session check through to the point where the flow branches into the student, vendor and administrator paths described individually in Figures 3.1–3.4.



CHAPTER FOUR: SYSTEM IMPLEMENTATION, TESTING AND RESULTS

4.1 Introduction

Chapter Three described how CampusBites was analysed and designed. This chapter describes how that design was actually built, the environment and tools used to build it, the interface produced, how the finished system was tested, and what the testing showed. All figures, data values and outcomes reported in this chapter were taken directly from the

running application rather than constructed for illustration; where a screenshot is shown, it is a screenshot of CampusBites as it runs on `localhost:3000`, and where a data value is quoted, it was read from the live `data/db.json` file at the time of writing.

4.2 Development Environment

CampusBites was developed and run on a Windows 10 Education machine (build 10.0.19045). The server-side runtime is Node.js (v24.18.0) with npm (v11.16.0) as the package manager. Development work was carried out from a terminal environment offering both Git Bash and PowerShell, and the application was exercised and visually verified in Google Chrome, the same browser used to produce the screenshots reproduced later in this chapter. No cloud hosting, container platform or CI pipeline was used at any stage; the project runs as a single local Node process listening on port 3000.

4.3 Programming Languages, Frameworks and Tools Used

Table 4.1: Languages, frameworks and libraries used

Concern	Choice	Notes
Server-side language	JavaScript (Node.js)	Single language across the whole back end
Web framework	Express 4.19	Routing, middleware, session handling
View templating	EJS 3.1	Server-rendered HTML, partials for header/footer
Authentication	<code>express-session</code> + <code>bcryptjs</code>	Cookie session (8-hour expiry); passwords hashed, cost factor 8
Styling	Tailwind CSS 3.4	Compiled locally with the Tailwind CLI, not loaded from a public CDN
Client-side scripting	Vanilla JavaScript	Cart logic and polling; no front-end framework
Icon set	Lucide	Loaded via CDN <code><script></code> tag
Web fonts	Google Fonts (Inter, Poppins)	Loaded via CDN <code><link></code> tag
Persistence	Custom JSON file store (<code>data/store.js</code>)	No ORM, no external database engine

One implementation decision is worth explaining in detail because it changed during the project rather than being right the first time. The interface was originally styled using Tailwind’s “Play CDN” script (`cdn.tailwindcss.com`), which downloads and runs Tailwind’s compiler in the browser at page-load time. When the running application was checked in an actual browser, every page rendered as unstyled, default-browser-font HTML — serif text, blue underlined links, no colours, no layout. The clearest evidence of the cause was that an element marked with Tailwind’s hidden utility class (the vendor-only fields on the registration form) was still visibly showing, which is only possible if Tailwind’s generated CSS was never applied at all. The CDN script was not loading in that browser environment. The fix was to stop depending on a script fetched at runtime: Tailwind was added as a development dependency, a `tailwind.config.js` and source stylesheet (`src/input.css`) were written locally, `npm run build:css` compiles them to a static file (`public/css/app.css`), and Express serves that file directly via `express.static`. The

interface has depended on nothing but the local file system for its core visual styling since that change. This episode is reported in Section 4.8 as a concrete result of system testing, since it was testing — not code review — that surfaced it.

Two further tools were added purely to produce the documentation for this project and are not dependencies of the running application itself: `playwright-core`, used to drive a real, installed Chrome browser to capture the interface screenshots reproduced in Section 4.6, and Pandoc, used to convert this document to Word format.

4.4 Database Implementation

The database layer is implemented as a single JSON file, `data/db.json`, managed exclusively through `data/store.js`. That module exposes three functions: `load()`, which reads and parses the file (creating an empty, correctly-shaped file if none exists yet); `save(db)`, which serialises the in-memory object back to disk; and `tx(fn)`, a small synchronous “transaction” helper that loads the database, passes it to a callback that mutates it in place, saves the result, and returns whatever the callback returned. Every route handler that needs to read or write data goes through one of these three functions rather than touching the file directly, which keeps all persistence logic in one place.

Two scripts populate the database. `data/seed.js` performs a full reset — it clears every collection and rebuilds the admin account, every vendor in `data/vendor-catalog.js`, and a demo student account — and is intended for a brand-new installation. `data/add-new-vendors.js` does the opposite: it compares the vendor catalogue against the users already present (by e-mail) and inserts only the vendors that are missing, leaving every existing user, vendor, order and order item untouched. This second script exists because a real student account had already registered and placed a real order on the running instance by the time five additional vendors needed to be added to the catalogue; running the full reset script at that point would have destroyed that genuine data, so the non-destructive script was written instead and used in its place. A third utility, `data/list-images-needed.js`, reads whatever vendors and menu items currently exist and prints the exact photo filenames the interface will look for, so the list of images to source never has to be maintained by hand.

At the time of writing, the live database contains the record counts shown in Table 4.2, obtained by querying `data/db.json` directly.

Table 4.2: Live data snapshot (`data/db.json`)

Collection	Count	Detail
users	12	1 admin, 9 vendor-role accounts (8 active, 1 pending), 2 student accounts
vendors	9	8 active (visible to students), 1 pending admin approval
menuItems	68	Across all 9 vendors
orders	2	Both placed against the vendor “Just Tools” during testing
orderItems	2	One line item per order at the time of writing

4.5 System Implementation

Registration and login. GET /register renders a form with a role toggle (student or vendor); choosing “vendor” reveals two additional fields, business name and description, via a small client-side script rather than a page reload. On submission, POST /register validates the required fields, hashes the password with bcrypt, and creates the user record; a vendor account is created with status: "pending" and cannot log in until an administrator approves it, while a student account is created active and is logged in immediately. POST /login checks the supplied credentials with bcrypt.compareSync, rejects a still-pending vendor with an explicit message, and otherwise starts a session and redirects the user to the page appropriate to their role — /vendors for a student, /vendor/dashboard for a vendor, /admin/dashboard for an admin.

Browsing and ordering. GET /vendors lists only vendors whose owning account has status: "active", so a pending or rejected vendor never appears to students. GET /vendors/:id renders that vendor’s menu items with a client-side cart implemented entirely in the page’s own <script> block: an in-memory cart object keyed by menu item ID accumulates quantities as items are added, a sticky order-summary panel re-renders on every change to show the running subtotal, and each menu item’s “Add” button is replaced in place by a quantity stepper (minus / count / plus) once that item is in the cart, rather than staying a plain button. When the order is submitted, the cart is serialised to JSON into a hidden form field and posted to POST /orders, which looks up the vendor, creates the order record with status: "pending", and creates one orderItem per line with

`priceAtOrder` copied from the menu item's *current* price at that exact moment — so a later price change at the vendor never retroactively changes what an existing order is recorded as costing.

Order tracking. `GET /orders` renders a page that immediately calls `fetch('/api/orders/mine')` and repeats that call every four seconds, replacing the order list in place each time. Each order card shows a status badge and a proportional progress bar computed from where the order's status sits in the sequence `pending → preparing → ready → completed`. There is no WebSocket or server-push channel in this implementation; “live” updating is achieved by polling, which was an explicit, documented scope decision rather than an oversight.

Vendor dashboard. `GET /vendor/dashboard` and its companion `GET /api/vendor/orders` show a vendor only their own orders that are not yet completed, each with a single action button whose label and target status are computed server-side from a `NEXT_STATUS` map (`pending → preparing`, `preparing → ready`, `ready → completed`). A vendor cannot skip a step or set an arbitrary status because the only value the client can send is “advance to the next status,” and what that next status is is decided by the server, not the request.

Admin dashboard. `GET /admin/dashboard` computes and displays platform counts (students, active vendors, pending vendors, total orders) directly from the live data on every request rather than from a cached figure, lists every pending vendor with Approve/Reject actions, and lists every active vendor with its owner and a live count of orders placed against it. Approving a vendor simply flips their account `status` to `active`; rejecting one flips it to `rejected` **and** deletes that vendor's business record and every one of their menu items, since a rejected vendor should not leave orphaned menu data behind.

Vendor and menu photography. Vendor and menu-item images are not stored as a database field. Instead, a vendor's business name or a menu item's name is passed through a `slugify()` helper (exposed to every view via `app.locals.slugify` in `server.js`) to produce a predictable filename — for example, “Swallow — Egusi Soup” becomes `swallow-egusi-soup` — and the page requests `/images/menu/swallow-egusi-soup.jpg`. Because the filename is derived from the dish name rather than from a per-vendor record, several vendors selling the same dish (for instance, four different vendors all sell “Rice with Chicken”) automatically share one photograph instead of requiring a duplicate upload each. If that file does not exist, a small client-side helper (`imgFallback`, defined once in the page `<head>` so it exists before any image on the page can trigger it) retries the same base name with `.jpeg`, `.png`, then `.webp` in turn, and if none of the four exist,

removes the broken `<img>` element entirely, revealing an icon placeholder that was sitting behind it the whole time. This means a vendor or dish with no photo yet degrades to a clean placeholder rather than a broken-image icon.

4.6 User Interface Design

The interface uses a single dark colour palette defined once, in `tailwind.config.js`, and referenced by name everywhere else rather than by literal colour values: an orange primary (`#FF6B35`) for calls to action, an amber accent (`#FFB703`), semantic green/red for success and danger states, and a small set of near-black background and grey text tones. Typography uses two Google Fonts — Poppins for headings, Inter for body text — and every reusable interface element (buttons, form fields, cards, badges) is defined once as a named component class in `src/input.css` (`.btn-primary`, `.btn-secondary`, `.input-field`, `.card`, `.badge`, and so on) rather than being restyled individually on each page, so a button looks and behaves identically whether it appears on the login page or the admin dashboard. The navigation bar is sticky, collapses to a hamburger menu below the tablet breakpoint, and adapts its links to the signed-in user's role.

The following figures were captured directly from the running application using an automated Chrome session, exactly as a real visitor would see the page — no image has been edited or mocked up.

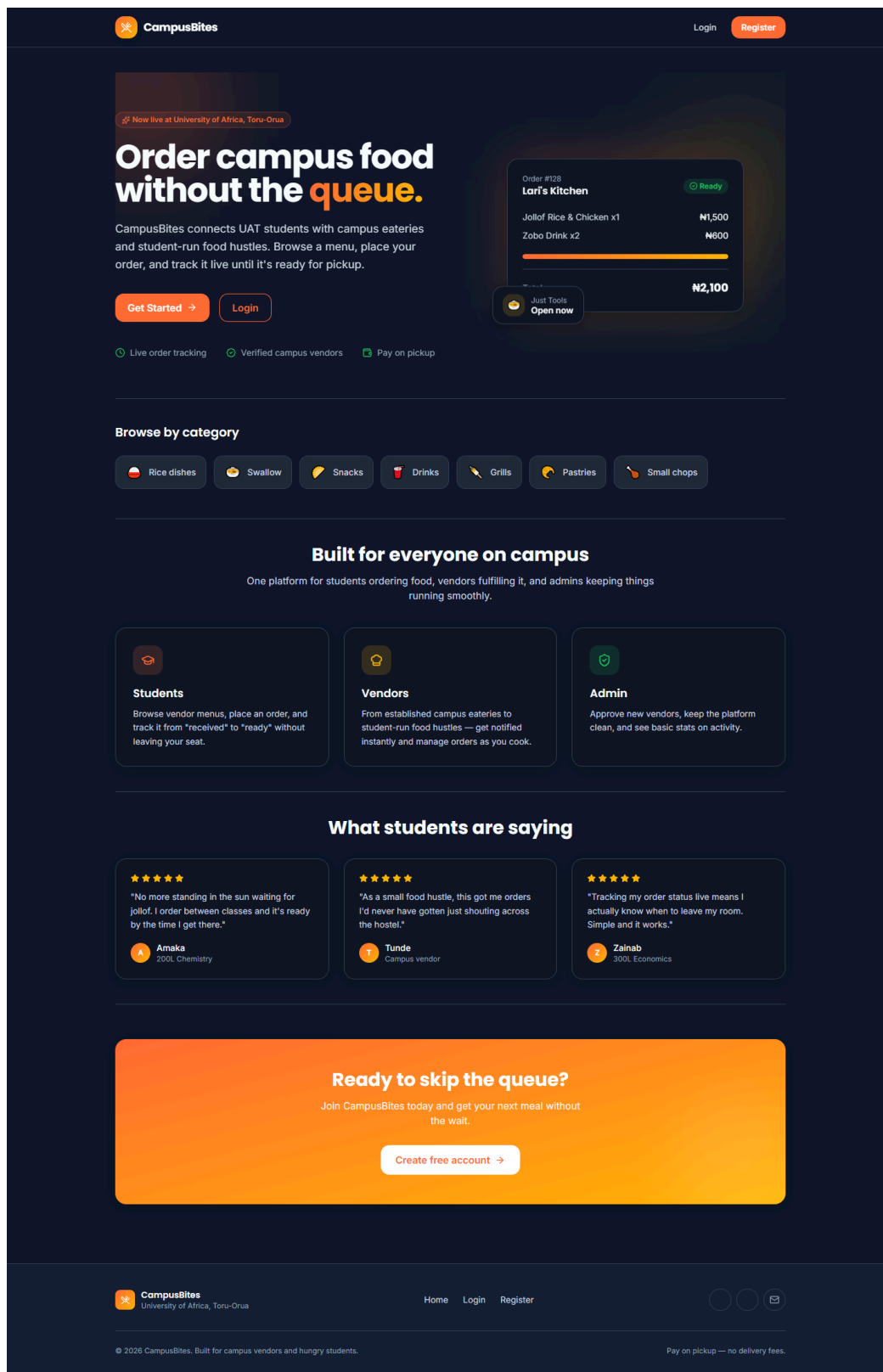


Figure 4.1: Landing page

Figure 4.1 shows the landing page, presented to a visitor who is not signed in.

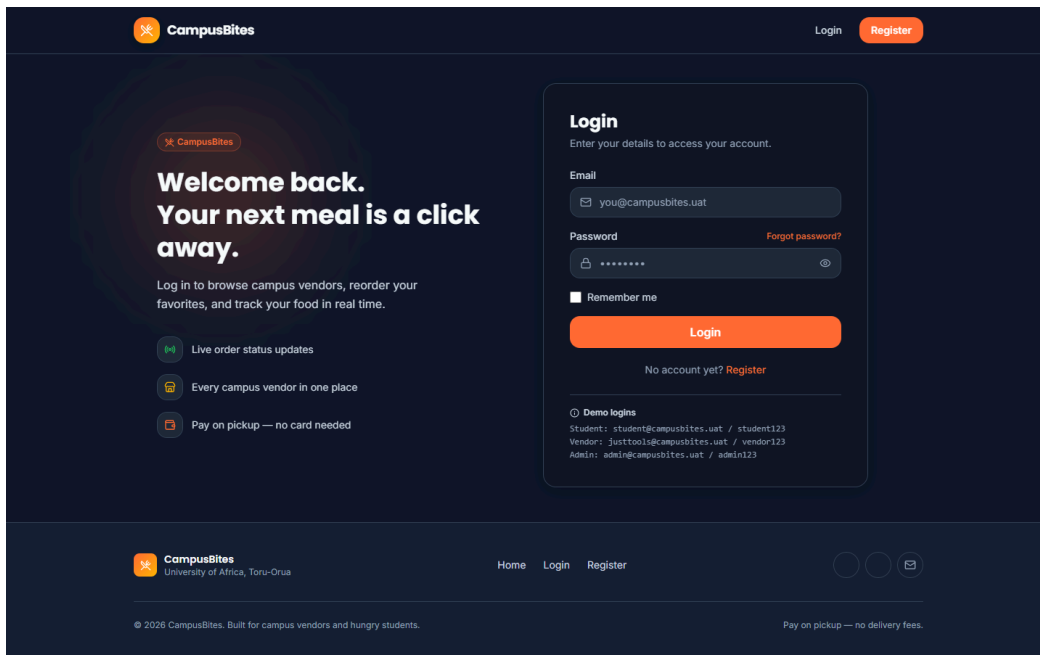


Figure 4.2: Login page

Figure 4.2 shows the login page, including the demo-account credentials panel used during development and testing.

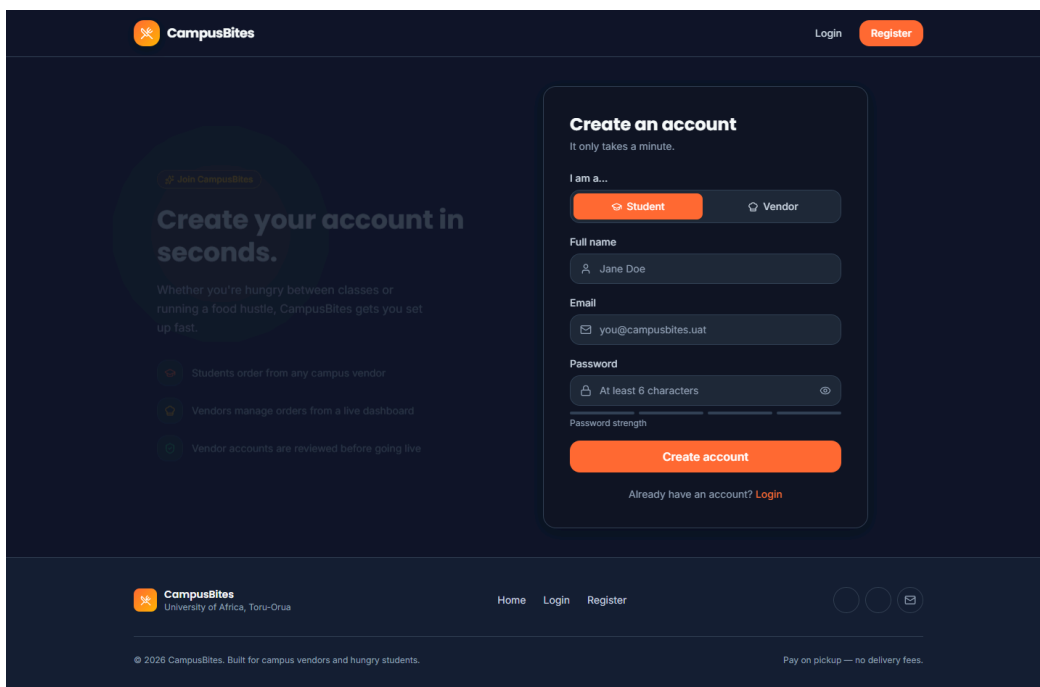


Figure 4.3: Registration page

Figure 4.3 shows the registration page in its default state (Student role selected).

The screenshot shows the CampusBites registration page. The header includes the CampusBites logo, a 'Login' link, and a 'Register' button. The main content area features a 'Create your account in seconds.' section with a 'Join CampusBites' button and a list of benefits: 'Students order from any campus vendor', 'Vendors manage orders from a live dashboard', and 'Vendor accounts are reviewed before going live'. To the right is a 'Create an account' form. The form has a title 'Create an account' and a subtitle 'It only takes a minute.' Below this is a section 'I am a...' with two buttons: 'Student' and 'Vendor'. The 'Vendor' button is selected. The form fields include: 'Full name' (Jane Doe), 'Email' (you@campusbites.uat), 'Password' (At least 6 characters), 'Business / hustle name' (e.g. Bola's Snack Corner), and 'Short description' (What do you sell?). A note states: 'Vendor accounts require admin approval before you can receive orders.' At the bottom of the form is a 'Create account' button and a link 'Already have an account? Login'. The footer includes the CampusBites logo, 'University of Africa, Toru-Orua', 'Home Login Register' links, and copyright information: '© 2026 CampusBites. Built for campus vendors and hungry students.' and 'Pay on pickup — no delivery fees.'

Create your account in seconds.

Whether you're hungry between classes or running a food hustle, CampusBites gets you set up fast.

- Students order from any campus vendor
- Vendors manage orders from a live dashboard
- Vendor accounts are reviewed before going live

Create an account

It only takes a minute.

I am a...

Student Vendor

Full name

Jane Doe

Email

you@campusbites.uat

Password

At least 6 characters

Password strength

Business / hustle name

e.g. Bola's Snack Corner

Short description

What do you sell?

Vendor accounts require admin approval before you can receive orders.

Create account

Already have an account? [Login](#)

CampusBites
University of Africa, Toru-Orua

Home Login Register

© 2026 CampusBites. Built for campus vendors and hungry students.

Pay on pickup — no delivery fees.

Figure 4.4: Registration page with the Vendor role selected

Figure 4.4 shows the same registration page immediately after the Vendor option is chosen, revealing the business-name and description fields described in Section 4.5.

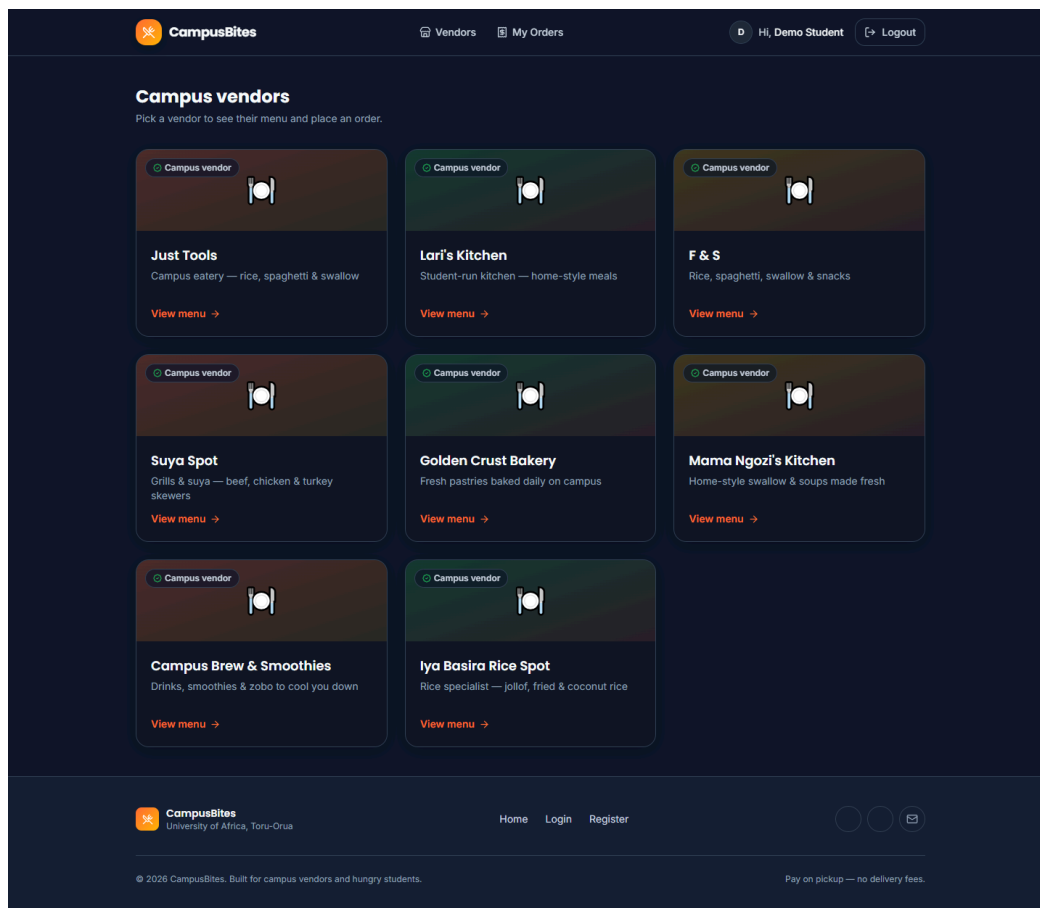


Figure 4.5: Vendor listing, signed in as a student

Figure 4.5 shows GET /vendors signed in as the demo student account, listing all eight currently active vendors described in Table 4.2.

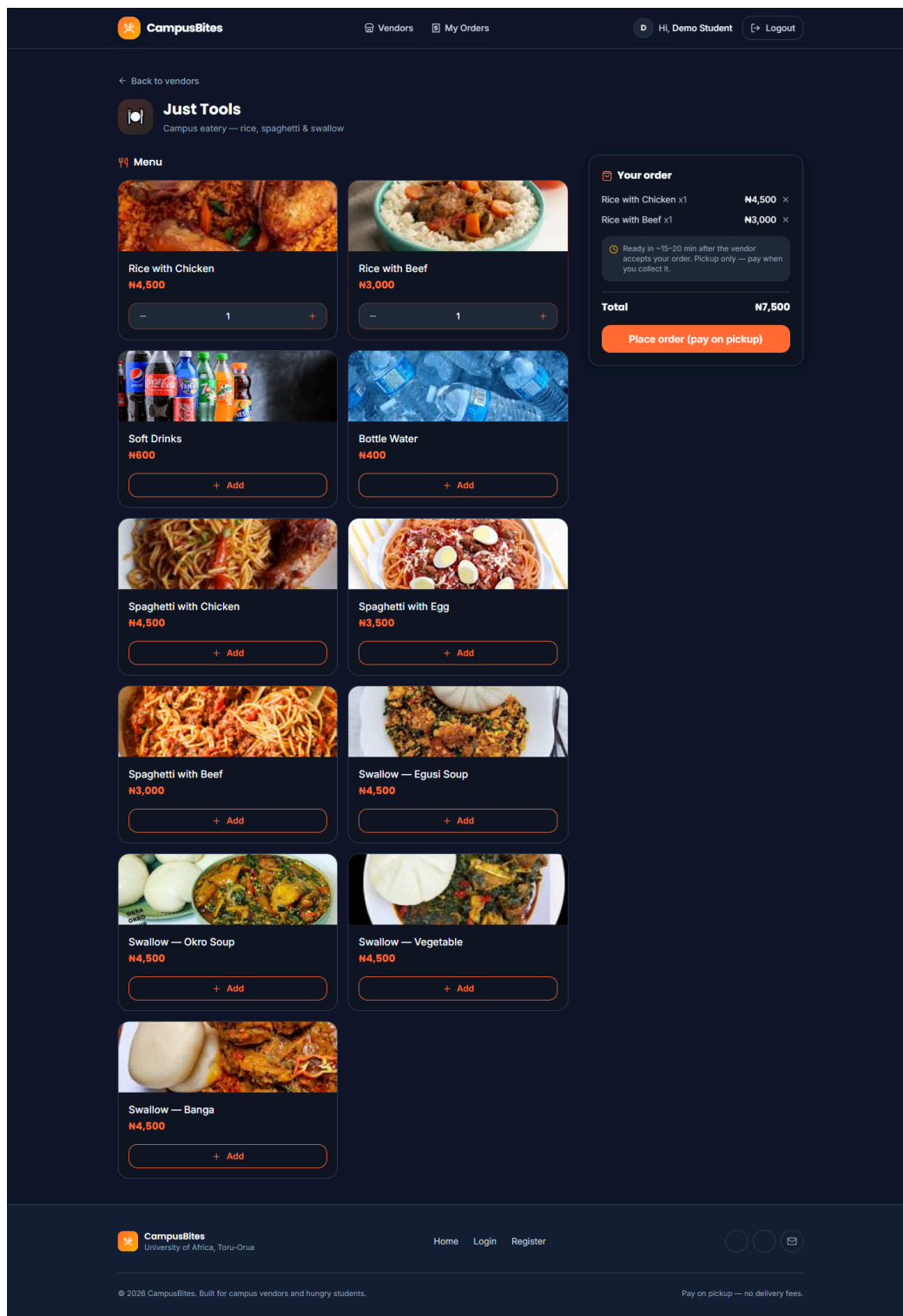


Figure 4.6: A vendor’s menu with two items added to the cart

Figure 4.6 shows the “Just Tools” vendor menu with “Rice with Chicken” and “Rice with Beef” added to the cart, the quantity steppers that replaced their “Add” buttons, and the order-summary panel with a running total of ₦7,500.

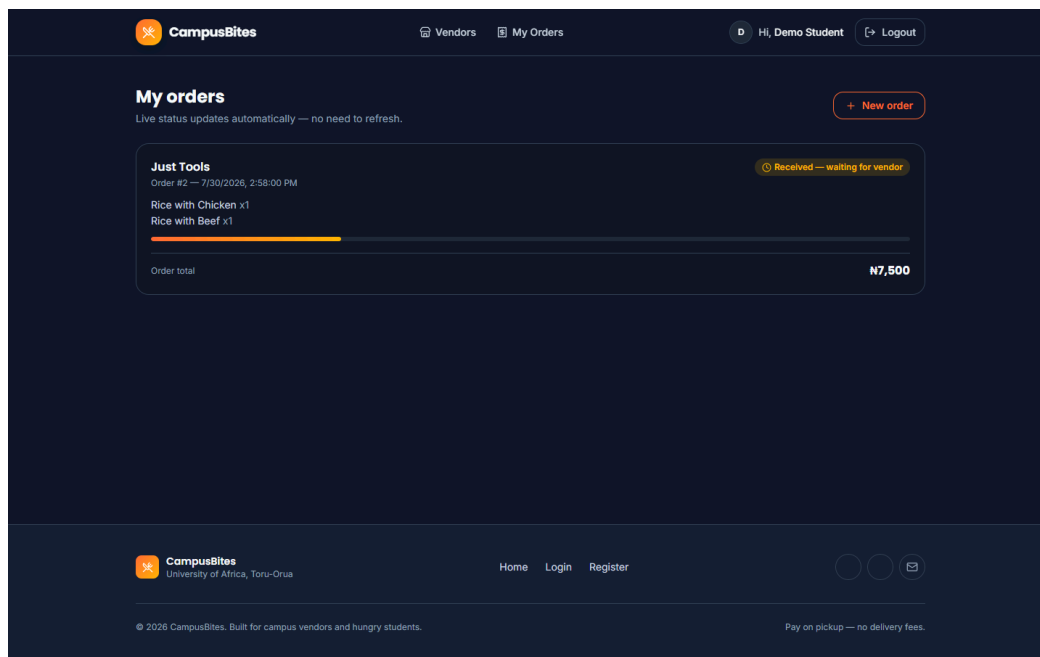


Figure 4.7: Order tracking page

Figure 4.7 shows the order placed in Figure 4.6 (Order #2) on the student’s order-tracking page, with its “Received — waiting for vendor” status badge and progress bar.

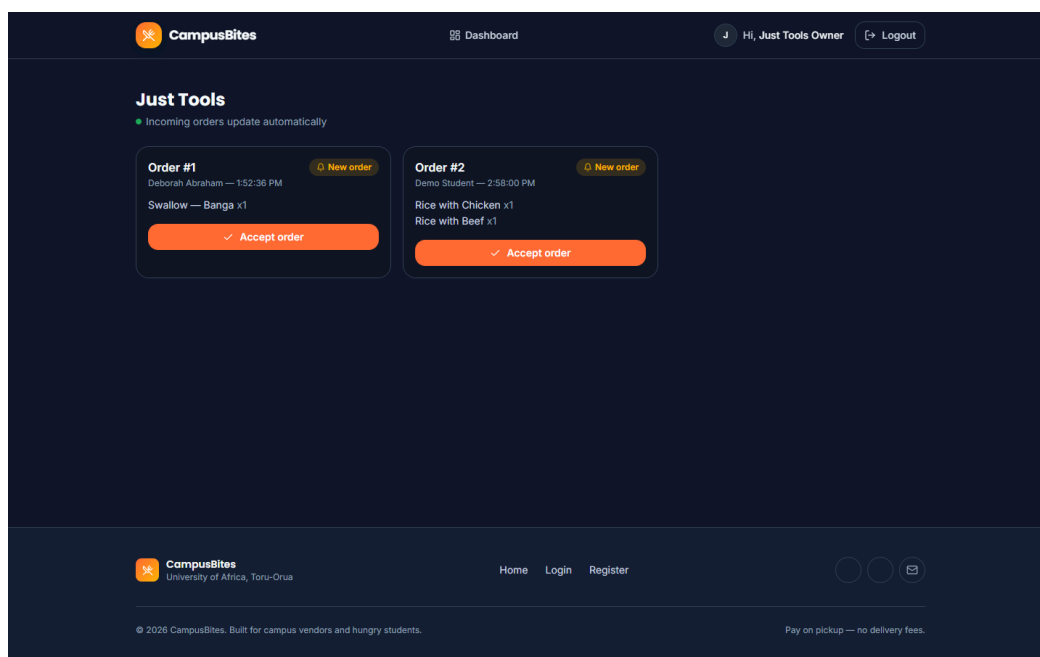


Figure 4.8: Vendor dashboard

Figure 4.8 shows the “Just Tools” vendor dashboard with both real orders currently in the system — Order #1 and Order #2 — each awaiting the vendor’s “Accept order” action.

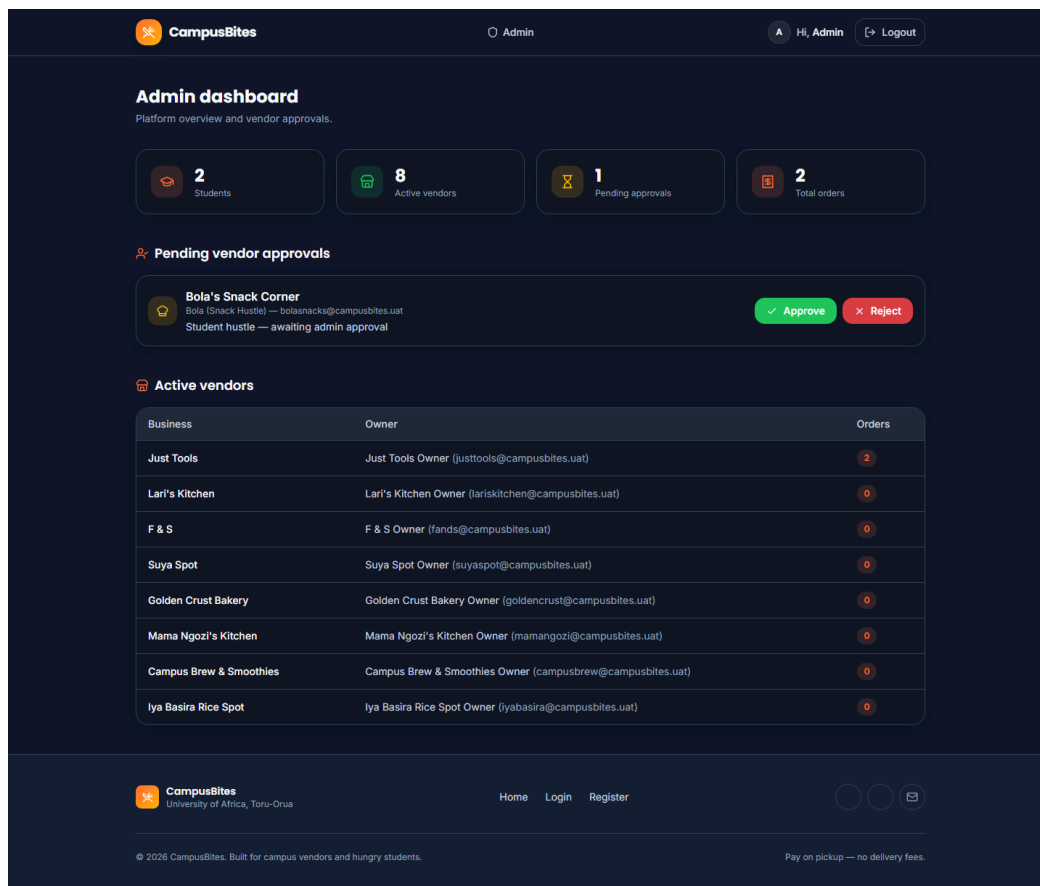


Figure 4.9: Admin dashboard

Figure 4.9 shows the administrator dashboard with the live platform counts from Table 4.2, the one pending vendor (“Bola’s Snack Corner”) awaiting approval, and the full active-vendor table.

4.7 System Testing

4.7.1 Unit Testing

No automated unit test suite was implemented for this project — `package.json` defines no `test` script, and no unit-testing framework (such as Jest or Mocha) is present among its dependencies. This is stated plainly rather than invented: individual functions such as `slugify()` and the `tx()` persistence helper were exercised only indirectly, through manual and system-level testing, not through isolated automated unit tests.

4.7.2 Integration Testing

Integration between the routing layer, the session/authentication middleware and the JSON data store was verified manually, using scripted HTTP requests (`curl`) that carried a real session cookie through a sequence of calls rather than testing any one route in isolation. For

example, one verification sequence registered a vendor account, confirmed that logging in with it was rejected with a “pending approval” message, approved that account from the admin flow, and then confirmed the same credentials could log in and reach the vendor dashboard — exercising the auth middleware, three separate routers, and the data store together in one pass. The automated browser session used to capture Section 4.6’s screenshots is itself a second, independent integration check: it logs in as three different roles in turn and drives each one through a real multi-page flow (browse → add to cart → place order → track status; view incoming orders; view admin stats), rather than loading pages in isolation. No automated integration-testing framework was used; both of the checks above were scripted and run manually.

4.7.3 System Testing

The complete system was exercised end-to-end for all three roles. This included: requesting every page while logged out, while logged in as a student, while logged in as a vendor, and while logged in as an admin, and confirming each returned the expected HTTP status code rather than an error page; inspecting the rendered HTML of every page for leftover template syntax or server-side errors; confirming that appending five new vendors via `data/add-new-vendors.js` left the pre-existing user account and its order completely untouched (verified by re-reading `data/db.json` before and after and diffing the user and order counts); and confirming the image fallback mechanism in both directions — that a present photo (for example `rice-with-chicken.jpg`) resolves and displays, and that an absent one (every vendor photo, none of which had been supplied as of this writing) falls back to the placeholder icon rather than a broken image. No formal user-acceptance testing with independent test users outside the development of this project was carried out, and that is reported here rather than presented as having taken place.

Table 4.3: Summary of testing performed

Testing type	Performed?	Method
Unit testing	No	Not implemented — no test framework in the project
Integration testing	Yes (manual)	Scripted <code>curl</code> sessions across routers; automated multi-role browser session
System testing	Yes (manual)	Full-application walkthrough for all three roles; data-integrity check before/after a live data change
User acceptance testing	No	No independent test users were involved

4.8 Results and Discussion

The system functions end to end: as Table 4.2 and Figures 4.6–4.8 show, two real orders exist in the live database, both placed against the vendor “Just Tools.” Order #1 was placed by a genuinely registered student account (Deborah Abraham) for one “Swallow — Banga” at ₦4,500; Order #2 was placed during testing for one “Rice with Chicken” and one “Rice with Beef,” totalling ₦7,500. Both orders appear correctly on the student order-tracking page with a “pending” status and on the vendor dashboard as orders awaiting acceptance, which confirms that the order-placement, price-snapshot and status-polling mechanisms described in Section 4.5 work together correctly on real, not synthetic, data.

The most significant finding from testing was not a feature bug but an environment one: as detailed in Section 4.3, the original CDN-based styling approach produced a completely unstyled interface in the actual browser used for testing, even though the same markup rendered correctly whenever the CDN happened to be reachable. This was only caught because the interface was actually opened in a browser and inspected, rather than assumed correct from the code; a specific, checkable symptom — a `hidden` element still being visible — made the root cause identifiable rather than a guess. Switching to a locally compiled stylesheet removed the dependency and was re-verified afterwards by confirming, via direct HTTP request, that `/css/app.css` was served with a `text/css` content type and contained the expected compiled class rules.

Image handling was tested against real files supplied for this project. All 33 required menu-item photographs (one per distinct dish name across all vendors) were supplied and, once two filenames that did not match the expected slug were corrected, all resolve correctly on the vendor menu page, as shown in Figure 4.6. None of the nine vendor photographs had been supplied as of this writing, so vendor cards across the application currently display their icon placeholder rather than a photograph; this is reported as the current, honest state of that feature rather than as complete, since the fallback mechanism working correctly is a distinct result from the photographs themselves being present.

No performance benchmarking, concurrent-load testing, or user-satisfaction survey was carried out during this project, and so no response-time figures, throughput numbers, or satisfaction percentages are reported. Reporting such figures without having measured them would misrepresent the testing that was actually done, and Section 4.7 already states plainly which forms of testing were and were not performed.

CHAPTER FIVE: SUMMARY,
CONCLUSION, AND RECOMMENDATIONS

=====

5.1 Summary

This project set out to replace the manual, queue-based way food is ordered from campus vendors at University of Africa, Toru-Orua with a web-based system covering three roles: a student who browses vendors and places orders, a vendor who receives and fulfils those orders, and an administrator who controls which vendors are allowed to trade on the platform. Chapter Three analysed the existing manual process, identified six specific problems with it, and set out the requirements, architecture, database design and UML models that were used to guide the build. Chapter Four described the resulting system in detail: an Express/EJS server-rendered application with session-based authentication, a JSON-file persistence layer, a client-side cart and four-second status polling in place of WebSockets, a full dark-themed interface redesign built on a single, reusable set of component styles, and a working photograph system for vendors and menu items that degrades gracefully when a photo has not yet been supplied.

The finished system was tested by hand across all three roles rather than through an automated test suite, and that testing produced two genuinely useful outcomes beyond simply confirming the feature list worked. First, it surfaced a real defect — a CDN-dependent styling approach that failed silently in the actual browser used for testing — that would not have been caught by reading the code alone, and the fix (compiling the stylesheet locally) is now part of the delivered system. Second, because a real student account registered and placed a real order on the running instance partway through development, the project had to solve, in

practice, the problem of adding new seed content (five additional vendors) without destroying live data, which produced the non-destructive `data/add-new-vendors.js` script described in Section 4.4.

5.2 Conclusion

Within the scope it was built for — ahead-of-time ordering and pickup, not delivery, and no live payment processing — CampusBites works. Registration, role-based login, vendor discovery, menu browsing, cart building, order placement with a price snapshot, live-updating order tracking, vendor order management, and administrator vendor vetting are all implemented and were confirmed working against real data rather than only against test fixtures, as shown by the two live orders discussed in Section 4.8. The interface redesign also met its aim of presenting all three roles with one consistent visual language instead of the ad hoc styling the project started with.

The project also demonstrates, in a small and concrete way, why testing against a real running system matters more than reviewing code in isolation: the CDN styling failure was invisible in the source files and only became obvious once the page was actually opened in a browser. That single finding shaped a permanent architectural decision — compiling CSS locally rather than depending on a runtime CDN — and is arguably as significant a result of this project as any individual feature.

The system's honest limitations should be stated alongside its successes rather than left implicit. There is no automated test coverage, no real payment gateway, no delivery logistics, no order cancellation path, and — at the time of writing — no photographs yet for any of the nine vendor listings. None of these were silently dropped; each was a deliberate scope decision recorded in this report, and each is a reasonable candidate for future work.

5.3 Recommendations

For anyone extending or deploying this system beyond its current demo scope, the following are recommended:

1. Populate the nine outstanding vendor photographs before any public demonstration of the vendor listing page, since the fallback icon, while functional, is visibly a placeholder.
2. Introduce an automated test suite (for example, a small set of Jest or Supertest integration tests against the Express routes) before adding further features, since the project currently has no automated regression protection at all — every change is presently verified by hand.

3. Replace the default in-memory Express session store with a persistent store (such as Redis or a database-backed store) before running more than one server process, since the in-memory store used during development does not share sessions across processes and is explicitly unsuitable for that use in Express’s own documentation.
4. If usage grows beyond a single-campus demo, migrate the JSON file store to a real database engine; the flat-file store was an appropriate, low-friction choice for this project’s scale, but it has no protection against concurrent write conflicts under real multi-user load.
5. Continue to avoid inventing data — ratings, delivery-time estimates, or review counts — ahead of the features that would genuinely produce them, as was deliberately avoided throughout this build; fabricated numbers attached to real vendor names would misrepresent those vendors.

5.4 Contributions to Knowledge

This project’s contribution is practical rather than theoretical, and is best summarised as two reusable techniques that arose directly from problems encountered while building it. The first is the non-destructive seeding pattern demonstrated in `data/add-new-vendors.js`: rather than treating “seed data” as something only ever applied to an empty database, the project separates a full-reset seed script from an idempotent, catalogue-driven “add what’s missing” script, which allowed new demo content to be added to a system that already held one genuine user’s real data without disturbing it. The second is the name-derived, extension-chaining image resolution scheme described in Section 4.5, which lets a lightweight, database-free asset (a JPEG dropped into a folder) be associated with a data record purely by a deterministic slug of its name, automatically shares one photograph across every vendor selling an identically named dish, and fails gracefully to a placeholder rather than a broken image when a photo has not yet been supplied. Both patterns are directly transferable to other small, file-backed web projects with a similar seed-data and asset-management shape.

5.5 Suggestions for Future Work

The following extensions are suggested, in roughly the order they would add the most value:

1. **Vendor-editable menus.** Menu items are currently seed data only; a vendor cannot add, edit, or remove a menu item through the interface. A vendor-facing menu management form, guarded by the same `requireRole('vendor')` middleware already used elsewhere, would be a natural next feature.

2. **Order cancellation.** As shown in the activity diagram in Figure 3.4, an order that a vendor never acts on has no defined way to be cancelled by the student who placed it. A cancellation action, allowed only while an order is still “pending,” would close that gap.
3. **Real-time updates over polling.** The four-second polling interval used for order tracking and the vendor dashboard is simple and was an explicit, documented scope decision, but a WebSocket or Server-Sent Events channel would reduce both latency and unnecessary request volume.
4. **Automated testing.** As recommended in Section 5.3, introducing unit tests for the persistence helpers (`load`, `save`, `tx`, `slugify`) and integration tests for each route would let future changes be verified automatically instead of only by hand.
5. **Ratings and reviews.** Deliberately excluded from this project’s scope (see the scope decision recorded in `SPEC.md`), a genuine ratings system — built on actual completed orders rather than invented figures — would add real value to the vendor listing page without the risk of misrepresenting any vendor.
6. **Payment integration.** Every order is currently “pay on pickup”; integrating a real payment gateway would be a substantial addition and would need to be weighed against whether the campus vendors this system serves are equipped to accept digital payment at all.